

TEMA 3. Excepciones

1. Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función `raiz` ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

En el lenguaje C, al carecer de un mecanismo nativo de excepciones, la gestión de errores se basa en la comunicación mediante el flujo de datos convencional. Para que la función `raiz` informe de un error al exterior sin imprimir mensajes directamente, se debe recurrir a convenciones de diseño donde el valor de retorno o parámetros adicionales actúen como indicadores de estado.

A continuación, se presentan dos formas comunes de diseñar este control de errores:

Opción 1: Uso de un valor de retorno reservado (Centinela)

Esta técnica consiste en devolver un valor que sea legal según el tipo de dato (`float`), pero que sea lógicamente imposible como resultado de la operación. En el caso de una raíz cuadrada, dado que el resultado nunca puede ser negativo, se puede devolver un número negativo (como `-1.0`) o la constante `NAN` (Not a Number) definida en la librería `math.h` para indicar que la entrada fue inválida.

El programa que llama a la función debe comprobar sistemáticamente este valor antes de proceder con el uso del dato. Es el método más sencillo, pero requiere que exista un rango de valores "imposibles" que puedan sacrificarse para representar el error, algo que no siempre es posible en todas las funciones matemáticas.

```
#include <stdio.h>
#include <math.h>

float raiz(float n) {
    if (n < 0) {
        return -1.0; // Valor centinela para indicar error
    }
    return sqrt(n);
}

int main() {
    float num = -5.0;
    float res = raiz(num);

    if (res == -1.0) {
        printf("Error: No se puede calcular la raiz de un numero negativo.\n");
    } else {
        printf("Resultado: %f\n", res);
    }
    return 0;
}
```

}

Opción 2: Paso de parámetros por referencia para el estado

En este diseño, la función devuelve un código de error (habitualmente un entero) y el resultado real de la operación se "devuelve" a través de un puntero pasado como argumento. Esta aproximación es más robusta, ya que separa claramente el **estado de la ejecución** (éxito o fallo) del **valor calculado**.

Si la función devuelve 0, se asume éxito; si devuelve un código distinto de cero, el valor almacenado en el puntero debe ignorarse. Este método es el estándar en muchas librerías de sistemas operativos y permite devolver múltiples tipos de errores de forma descriptiva, sin comprometer el rango de valores del resultado matemático.

```
#include <stdio.h>
#include <math.h>

// Devuelve 0 si es correcto, 1 si hay error
int raiz_segura(float n, float *error) {
    if (n < 0) {
        *error=1 // Código de error
        return 0;
    }
    *error=0;
    return sqrt(num);
}

int main() {
    int error=0;
    float num = -9.0;
    float resultado=raiz(num,&error);
    if (error!=0){
        printf("Error: No se puede calcular la raiz de un numero negativo.\n");
    }else{
        printf("Resultado: %f\n", res);
    }
}
```

2. Brevemente ¿Qué es una "excepción"? ¿Con qué objetivo las usa un programador cuando implementa funciones o cuando las llama?

Una **excepción** es un objeto que representa un error o evento inesperado durante la ejecución. Surge en situaciones atípicas básicamente. A diferencia de C, donde el error es un simple número (como -1), aquí es una estructura con información detallada sobre qué falló y en qué línea de código ocurrió.

Al **implementar una función**, el programador las usa para delegar la responsabilidad. En lugar de decidir si imprimir un mensaje o abortar, la función lanza la excepción y deja que quien la invocó decida cómo

reaccionar. Esto mantiene la lógica del cálculo separada de la gestión de errores.

Al **llamar a una función**, el objetivo es la robustez. Las excepciones obligan al programador a considerar los casos de falla, evitando que un error pase desapercibido por olvidar un `if`. Permiten agrupar el manejo de múltiples errores en un solo bloque, manteniendo el código principal limpio y legible.

3. Reescribe el mismo ejemplo de raíz, pero en Java, metiendo ese método en una clase `Calculadora` y llama a dicho método desde el método `main`, mostrando cómo se puede controlar desde fuera.

En Java, para informar de un error desde una función sin usar valores de retorno especiales, se utiliza la palabra clave `throw` seguida de un objeto de excepción. En este caso, se emplea `IllegalArgumentException`, una excepción predefinida para indicar que un parámetro no es válido (como un número negativo para una raíz).

Al llamar al método desde el `main`, el código se encierra en un bloque `try`. Si la función lanza la excepción, el flujo se desvía inmediatamente al bloque `catch`, donde se captura el objeto de error. Esto permite que el programa informe al usuario y continúe su ejecución en lugar de finalizar abruptamente por un error matemático.

```
public class Calculadora {

    public static double raiz(double n) {
        if (n < 0) {
            // Se lanza la excepción para que el llamador la gestione
            throw new IllegalArgumentException("No se puede calcular la raíz de un
número negativo.");
        }
        return Math.sqrt(n);
    }

    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        double numero = -9.0;

        try {
            double resultado = calc.raiz(numero);
            System.out.println("El resultado es: " + resultado);
        } catch (IllegalArgumentException e) {
            // Se captura el error lanzado por el método raiz
            System.out.println("Error detectado: " + e.getMessage());
        }

        System.out.println("El programa continúa su ejecución normal.");
    }
}
```

Este enfoque separa la lógica de cálculo (dentro del método `raiz`) de la lógica de interfaz de usuario (el `println` del error en el `main`). A diferencia de C, el valor de retorno de `raiz` es siempre un número válido, y

el error viaja por un canal paralelo de comunicación.

4. ¿Qué es **"lanzar"** una excepción? ¿Qué es **"controlar"** o **"capturar"** una excepción? ¿Qué es que se **"propague"** una excepción? ¿Qué le va ocurriendo a las funciones en la pila de llamadas por donde se va propagando la excepción? ¿Las funciones que no la controlan se reanudan después de alguna forma? Explica con el mismo ejemplo anterior en Java de la raíz cuadrada.

Lanzar una excepción es el acto de señalar un error mediante la palabra clave **throw**. En el ejemplo de la raíz, cuando se detecta un número negativo, la función "crea" un objeto de error y lo expulsa de su flujo normal. Esto detiene inmediatamente la ejecución de esa función, enviando el objeto de error hacia arriba, a quien la haya invocado.

Capturar o controlar una excepción consiste en encerrar una llamada a un método dentro de un bloque **try-catch**. El **try** vigila si ocurre un error y, si sucede, el **catch** lo atrapa. Al capturarla, el programador toma el control del problema, evitando que el programa se detenga y permitiendo que la ejecución continúe de forma segura después del bloque de captura.

La **propagación** ocurre cuando una función lanza una excepción y nadie la captura en ese nivel. El error viaja "hacia atrás" por la pila de llamadas (del método **raíz** al **main**, y del **main** al sistema operativo). Si una función en la pila no tiene un bloque **catch**, esa función finaliza abruptamente en ese mismo instante, liberando sus variables locales pero sin completar el resto de sus instrucciones.

Las funciones que no controlan la excepción **no se reanudan jamás**. Una vez que la excepción atraviesa una función sin ser capturada, esa función se considera terminada por error. El flujo de ejecución solo se retoma en el punto exacto donde alguien finalmente use un **catch**. Si nadie la captura en toda la jerarquía de llamadas, la Máquina Virtual de Java detiene el programa por completo y muestra el error por consola.

5. ¿Qué ventajas tiene frente a C, la **"propagación natural"** de las excepciones a través de la pila (*stack*) de llamadas?

En C, la propagación de un error es **manual y obligatoria** en cada paso. Si una función **A** llama a **B**, y **B** llama a **raíz**, cada una de esas funciones debe incluir sentencias **if** para capturar el error de la anterior y volver a devolver un código de error hacia arriba. Si un programador olvida una sola comprobación en la cadena, el error se "silencia" y el programa continúa con datos inválidos, lo que suele derivar en fallos catastróficos difíciles de depurar.

La **propagación natural** de Java permite que el error "salte" automáticamente a través de múltiples niveles de la pila hasta encontrar a alguien capacitado para manejarlo. Esto significa que las funciones intermedias no necesitan código específico de gestión de errores si no pueden aportar una solución; simplemente se cierran y dejan que la excepción fluya. Esto reduce drásticamente el "ruido" en el código, permitiendo que la lógica principal sea mucho más limpia y directa.

Otra ventaja fundamental es la **imposibilidad de ignorar el error por descuido**. En C, si no se comprueba un valor de retorno, el programa sigue adelante. En Java, si una excepción se propaga y nadie la controla, el programa se detiene informando exactamente qué falló y dónde (el *stack trace*). Esto garantiza que los errores se hagan visibles de inmediato durante el desarrollo, en lugar de convertirse en comportamientos erráticos e inexplicables en producción.

Por último, esta jerarquía permite una **gestión centralizada**. Un programador puede escribir diez funciones que realicen operaciones matemáticas y controlarlas todas con un único bloque `try-catch` en el método principal. En lugar de diez comprobaciones manuales, se tiene un solo punto de control que decide cómo informar al usuario, simplificando enormemente el mantenimiento del software.

6. En orientación a objetos, ¿las excepciones suelen ser objetos? ¿Qué ventajas tiene esto en términos de encapsulación? ¿Podemos entonces crear excepciones personalizadas?

En Java, efectivamente, las **excepciones son objetos**. Esto significa que cada vez que ocurre un error, se instancia una clase específica (como `ArithmeticException` o `NullPointerException`). Al ser objetos, heredan de una jerarquía común cuya raíz es la clase `Throwable`, lo que permite tratarlas de forma genérica o específica según convenga en los bloques `catch`.

En términos de **encapsulación**, esta naturaleza de objeto permite que la excepción transporte consigo toda la información relevante del error sin "ensuciar" el valor de retorno de la función. Un objeto de excepción puede contener mensajes de texto, códigos de error numéricos, el estado de las variables en el momento del fallo e incluso la traza completa de la pila de llamadas (*stack trace*), manteniendo todos estos detalles empaquetados y protegidos dentro del objeto.

Por supuesto, es posible y muy recomendable **crear excepciones personalizadas**. Para ello, simplemente se debe crear una nueva clase que herede de `Exception` (para excepciones que el compilador obliga a revisar) o de `RuntimeException` (para errores de lógica). Esto permite al programador definir errores con nombres que tengan sentido en el dominio de su aplicación, como `SaldoInsuficienteException` o `UsuarioNoEncontradoException`.

Crear excepciones propias mejora la legibilidad del código, ya que quien llama a la función sabe exactamente qué tipo de problema de "negocio" ha ocurrido, en lugar de recibir un error genérico de Java. Además, al ser una clase propia, se le pueden añadir métodos o atributos adicionales para ayudar a la recuperación del error, manteniendo la coherencia con los principios de la orientación a objetos.

```
// Ejemplo de excepción personalizada
public class NumeroNegativoException extends RuntimeException {
    public NumeroNegativoException(double valor) {
        super("El valor " + valor + " no es válido para esta operación.");
    }
}
```

7. En relación con las ventajas de la encapsulación, comparando el ejemplo en C con Java. ¿Qué **información esencial** lleva cualquier **objeto excepción** que es muy útil tener cuando se llega a un manejador?

En el ejemplo de C, cuando se detecta un error, el manejador solo recibe un número o un puntero nulo. No hay rastro de qué función llamó a cuál, ni en qué línea exacta falló el programa. El objeto **excepción en Java**,

en cambio, encapsula una "fotografía" del estado del sistema en el instante del error, lo que permite una depuración mucho más precisa.

La información más valiosa que transporta es el **StackTrace** (traza de la pila). Este es un registro detallado de toda la jerarquía de llamadas que estaban activas: desde el **main**, pasando por las funciones intermedias, hasta llegar a la línea exacta de la clase **Calculadora** donde saltó el error. Sin esto, en proyectos grandes, sería casi imposible adivinar qué camino tomó el código para llegar al fallo.

Además, el objeto incluye un **mensaje descriptivo** y, opcionalmente, la **causa original** (si una excepción provocó otra). Al ser un objeto, el manejador en el **catch** puede interrogarlo mediante métodos como **e.getMessage()** o **e.printStackTrace()**. Esto permite que el programador registre en un archivo de log no solo que "algo falló", sino el contexto completo del problema, facilitando su corrección posterior.

Finalmente, gracias a la herencia, el objeto lleva consigo su propia **identidad de tipo**. Esto permite que un solo bloque de código pueda distinguir automáticamente entre un error de red, uno de base de datos o uno de lógica matemática (**IllegalArgumentException**), simplemente verificando la clase del objeto recibido. En C, esto requeriría complejas estructuras de control o códigos de error globales difíciles de gestionar.

8. En Java, sobre el bloque **"try-catch"**, ¿se pueden tener más de un bloque **catch**? ¿cuántos bloques **catch** se ejecutan?

Sí, en Java es perfectamente posible y muy común encadenar **múltiples bloques catch** después de un único **try**. Esto permite que un mismo fragmento de código sea vigilado ante diferentes tipos de fallos, gestionando cada uno de manera específica según su naturaleza. Por ejemplo, en una calculadora avanzada, podrías querer tratar de forma distinta un error de división por cero de un error de formato al introducir los números.

En cuanto a la ejecución, **solo se ejecuta un bloque catch** por cada excepción lanzada: el primero que coincida con el tipo de la excepción o que sea una clase padre de esta. En el momento en que ocurre el error dentro del **try**, Java busca de arriba hacia abajo el primer **catch** compatible; una vez que lo encuentra y ejecuta su código, el resto de los bloques **catch** asociados a ese **try** se ignoran por completo.

Debido a este comportamiento, el **orden de los bloques es crítico**. Si se coloca un **catch** de una clase muy genérica (como **Exception**) antes que uno de una clase específica (como **ArithmeticException**), el genérico atraparé el error siempre y el específico quedará "muerto" o inaccesible, lo que provocará un error de compilación. Siempre se debe ir de lo más particular a lo más general.

Esta estructura es mucho más limpia que en C, donde tendrías que anidar múltiples **if-else** para comprobar distintos códigos de error. Aquí, el lenguaje separa visualmente cada escenario de fallo, permitiendo que el programador defina una estrategia de recuperación distinta para cada problema técnico o de negocio que pueda surgir.

9. Si las excepciones producen rupturas en el código llamador, ¿cómo podemos garantizar que se ejecuta siempre finalmente un código necesario para cierre de ficheros, liberación de recursos, antes de que continúe propagándose la excepción? Pon un ejemplo en Java con **finally**, tanto con **catch** como sin él.

Para garantizar que un fragmento de código se ejecute sin importar lo que ocurra, Java proporciona el bloque **finally**. Este bloque se coloca después del **try** (y del **catch**, si lo hay) y tiene la propiedad de ejecutarse siempre: tanto si el código del **try** finaliza con éxito, como si se lanza una excepción que es capturada, o incluso si se lanza una excepción que **no** es capturada y sigue propagándose hacia arriba en la pila.

En lenguajes como C, si una función retorna anticipadamente debido a un error, el programador debe recordar liberar manualmente la memoria o cerrar los ficheros en cada punto de salida. Con **finally**, Java asegura que los recursos críticos (como una conexión a base de datos o un archivo abierto) se cierren correctamente, evitando fugas de memoria o bloqueos de archivos que dejarían el sistema inestable.

A continuación, se muestra un ejemplo donde se intenta realizar una operación. En el primer caso, usamos **catch** para gestionar el error; en el segundo, omitimos el **catch** para que la excepción se propague, pero aun así garantizamos el cierre del recurso.

```
public class GestorRecursos {
    public void operacionConFichero() {
        System.out.println("Abriendo fichero...");
        try {
            int resultado = 10 / 0; // Provoca una excepción
        } catch (ArithmeticException e) {
            System.out.println("Error capturado: División por cero.");
        } finally {
            // Este código se ejecuta SIEMPRE
            System.out.println("Cerrando fichero y liberando recursos...");
        }
    }

    public void operacionPeligrosa() {
        try {
            System.out.println("Ejecutando algo que fallará y no capturaré...");
            throw new RuntimeException("Error fatal");
        } finally {
            // Se ejecuta antes de que la excepción "salte" al método superior
            System.out.println("Limpieza de emergencia antes de propagar el
error.");
        }
    }
}
```

Es importante destacar que el bloque **finally** se ejecuta incluso si dentro del **try** o del **catch** existe una instrucción **return**. Java "pausa" ese retorno, ejecuta el contenido del **finally** y solo entonces permite que el flujo continúe su camino. Esta es la herramienta definitiva para mantener la integridad del sistema ante cualquier imprevisto.

10. En Java, el bloque **finally** puede ir sin **catch**? ¿Se ejecuta siempre tanto si ocurre como si no ocurre una excepción? ¿Y si hay un **return** en medio del **try**?

Sí, en Java es perfectamente válido utilizar un bloque **try-finally** sin incluir un bloque **catch**. Esta estructura se emplea cuando el método actual no sabe o no debe gestionar el error, dejando que la excepción se propague a una función superior, pero aun así necesita garantizar que ciertos recursos (como cerrar un descriptor de archivo en C) se liberen antes de abandonar el método.

El bloque **finally** tiene la garantía de ejecución más fuerte del lenguaje: **se ejecuta siempre**, tanto si el código del **try** finaliza con éxito como si ocurre una excepción (sea esta capturada por un **catch** o no). Es el mecanismo de seguridad que evita que el programa deje "cabos sueltos", como conexiones a bases de datos abiertas o archivos bloqueados, independientemente de los errores lógicos que ocurran.

En cuanto a la presencia de un **return**, el comportamiento de Java es muy estricto: el bloque **finally** se ejecuta incluso si hay una instrucción **return** dentro del **try** o del **catch**. Cuando la ejecución alcanza el **return**, Java "reserva" el valor que se va a devolver, pausa la salida de la función, ejecuta todo el código dentro del **finally** y, solo una vez terminado este, efectúa el retorno físico al llamador.

Este comportamiento garantiza que la limpieza de recursos sea prioritaria sobre la salida de la función. Para un programador que viene de C, esto elimina la necesidad de replicar código de limpieza antes de cada **return** ante errores, centralizando toda la lógica de finalización en un único punto seguro y automático.

11. En Java, qué son las excepciones "controladas" y las "no controladas"? ¿Qué papel juega `RuntimeException`? Pon un ejemplo de excepciones típicas controladas y no controladas que incluso nosotros mismos podríamos usar. Haz dos listas con 3 o 4 ejemplos de situación donde se suele preferir una excepción controlada y donde se suele preferir una excepción no controlada.

En Java, la diferencia fundamental entre excepciones radica en si el compilador nos obliga a gestionarlas o no. Las **excepciones controladas** (*Checked*) son aquellas que el compilador exige que captures con un **try-catch** o declares en la firma del método con **throws**. Representan situaciones que un programa bien escrito debería prever, como que un archivo no exista o un fallo de red.

Las **excepciones no controladas** (*Unchecked*) son aquellas que heredan de la clase **RuntimeException**. El papel de esta clase es agrupar errores que suelen ser fallos de lógica del programador. El compilador no obliga a tratarlas porque se supone que lo correcto es arreglar el código para que no ocurran, en lugar de intentar recuperarse de ellas durante la ejecución.

Ejemplos comunes

- **No controladas (`RuntimeException`):** `NullPointerException` (acceder a un puntero nulo), `ArrayIndexOutOfBoundsException` (índice de array fuera de rango) o `ArithmeticException` (división por cero).
 - **Controladas:** `IOException` (error de entrada/salida), `SQLException` (error de base de datos) o `FileNotFoundException` (archivo no encontrado).
-

Cuándo usar cada una

Se prefiere una excepción CONTROLADA cuando:

- El error es **contingente** y externo al programa (fallo en la conexión WiFi).
- El usuario del método **puede hacer algo** para recuperarse (pedir otra ruta de archivo).
- Se quiere garantizar por contrato que quien use el código **considere el fallo**.
- Situaciones de "negocio" que son probables (ej. saldo insuficiente al sacar dinero).

Se prefiere una excepción NO CONTROLADA cuando:

- El error es un **defecto de programación** (pasar un parámetro **null** donde no se debe).
- Es un error **irrecuperable** o muy genérico que detendría la lógica de todos modos.
- Se quiere evitar ensuciar el código con bloques **try-catch** excesivos por errores evitables.
- Situaciones que violan las **precondiciones** técnicas del método (ej. un índice negativo).

12. ¿Qué es y para qué se usa **throws**? ¿Por qué es alternativa a capturar una excepción controlada?

La palabra clave **throws** es una declaración en la firma de un método que indica que dicho método no manejará una excepción específica, sino que la "rebotará" a quien lo haya llamado. Funciona como un contrato de aviso: el programador le dice al compilador y a otros desarrolladores: "Este método puede fallar por esta razón, y es responsabilidad de quien lo use decidir qué hacer".

Se utiliza principalmente con las **excepciones controladas** (*Checked Exceptions*). Dado que el compilador de Java prohíbe ignorar estos errores, un método solo tiene dos opciones: capturar la excepción con un **try-catch** o declararla con **throws**. Si se elige esta última, el método se lava las manos y transfiere la obligación de gestionar el error al siguiente nivel de la pila de llamadas.

Es una **alternativa a capturar la excepción** porque permite que el manejo de errores se centralice en las capas superiores de la aplicación (como la interfaz de usuario) en lugar de dispersarlo por toda la lógica interna. Por ejemplo, si un método de bajo nivel que lee un archivo falla, a menudo es mejor que no intente arreglarlo él mismo, sino que lance la excepción hacia arriba para que el programa principal decida si debe pedirle al usuario otro nombre de archivo.

En resumen, **throws** permite la **propagación legal** de excepciones controladas. Sin esta cláusula, el código no compilaría si detecta un posible error de entrada/salida o de base de datos. Al usarla, se mantiene la seguridad del lenguaje (el error no será ignorado) pero se gana flexibilidad en el diseño del software al decidir exactamente en qué punto de la cadena de llamadas es más sensato actuar frente al problema.

```
// Ejemplo: El método no captura el error, lo delega usando throws
public void leerConfiguracion(String ruta) throws FileNotFoundException {
    File archivo = new File(ruta);
    Scanner lector = new Scanner(archivo); // Esto puede lanzar
    FileNotFoundException
}
```

13. Pon un ejemplo en Java de firma de método que incluya **throws**, de una función que abre un fichero pero que declara que no le interesa manejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del **finally**.

Cuando se utiliza la cláusula **throws**, el método está notificando formalmente que puede fallar y que no va a gestionar dicho error internamente. En el caso de abrir un fichero, la excepción **FileNotFoundException** es de tipo **controlada** (*checked*), por lo que el compilador obliga a decidir: o se captura con un **catch** o se declara en la firma para que el método llamador se haga responsable.

El uso de **finally** en este escenario es fundamental para la seguridad de los recursos. Aunque el método delegue la gestión del error "hacia arriba" mediante **throws**, tiene la obligación de cerrar cualquier recurso que haya llegado a abrir (como descriptores de memoria o flujos de datos) antes de que la excepción abandone el método y se propague por la pila de llamadas.

```
import java.io.*;
import java.util.Scanner;

public class LectorArchivos {

    // El método declara que puede lanzar la excepción sin capturarla
    public void leerDatos(String ruta) throws FileNotFoundException {
        File archivo = new File(ruta);
        Scanner sc = null;

        try {
            sc = new Scanner(archivo); // Aquí puede saltar la excepción
            while (sc.hasNextLine()) {
                System.out.println(sc.nextLine());
            }
        } finally {
            // Se ejecuta SIEMPRE: tanto si el fichero se leyó bien,
            // como si ocurrió el error y la excepción va de camino al main.
            if (sc != null) {
                sc.close();
                System.out.println("Recurso cerrado en el bloque finally.");
            }
        }
    }

    public static void main(String[] args) {
        LectorArchivos lector = new LectorArchivos();
        try {
            lector.leerDatos("config.txt");
        } catch (FileNotFoundException e) {
            // El main es quien finalmente decide qué hacer con el error
            System.out.println("Error en el main: El archivo no se encontró.");
        }
    }
}
```

Este diseño es muy robusto porque separa las responsabilidades. El método **leerDatos** se enfoca solo en su tarea (leer) y en su limpieza (**finally**), mientras que el **main** (o la capa de usuario) se encarga de la política de

errores. Es una evolución clara respecto a C, donde tendrías que gestionar el cierre del fichero y el retorno del error manualmente en cada paso.

14. ¿Podemos poner en `throws` excepciones no controladas, como `RuntimeException`? ¿Debería el método llamador entonces poner `try-catch` en ese caso? ¿Qué sentido tendría?

En Java, **es técnicamente posible** incluir excepciones no controladas (descendientes de `RuntimeException`) en la cláusula `throws` de un método. Aunque el compilador no obliga a hacerlo (a diferencia de las excepciones controladas), el lenguaje permite añadirlas como una forma de documentación explícita. Al hacerlo, se está indicando a otros programadores que, aunque el código sea sintácticamente correcto, existe un riesgo lógico específico que deben conocer.

En cuanto al método llamador, **no está obligado** por el compilador a utilizar un bloque `try-catch` aunque la excepción no controlada aparezca en el `throws`. Si el llamador decide ignorarla y la excepción ocurre, el programa simplemente se propagará por la pila de llamadas hasta encontrar un manejador o detener la ejecución. Esta es la principal diferencia con las *Checked Exceptions*, donde el `try-catch` o un nuevo `throws` serían estrictamente necesarios para que el código compile.

El sentido de incluir una `RuntimeException` en el `throws` es principalmente de **comunicación y diseño de API**. Sirve para advertir al usuario del método sobre precondiciones que no se están cumpliendo. Por ejemplo, si un método espera un número positivo y el usuario pasa uno negativo, incluir `throws IllegalArgumentException` en la firma ayuda a que herramientas de autocompletado y documentación (como Javadoc) alerten al desarrollador antes de que ejecute el programa.

Desde el punto de vista de la arquitectura, esto permite que el programador del método llamador decida conscientemente si el error es algo de lo que puede recuperarse (poniendo un `catch`) o si prefiere que el programa falle porque indica un error de programación grave. En C, esta intención se perdía en los comentarios del código; en Java, queda integrada en la propia estructura del lenguaje, aunque sea de forma opcional.

15. ¿Cuándo se recomienda usar excepciones controladas, como `IOException`, y cuándo no controladas como `IllegalArgumentException`? ¿Existen en todos los lenguajes ambas opciones? En los que sólo existe una opción, ¿cuál es la más habitual?

Las **excepciones controladas** se recomiendan para fallos que son **ajenos al control del programador** y que son razonablemente previsibles. Se trata de situaciones donde el programa interactúa con el "mundo exterior" (ficheros, redes, bases de datos). El objetivo es forzar al desarrollador a escribir un plan de contingencia (un bloque `catch`), garantizando que la aplicación no colapsará si, por ejemplo, un cable de red se desconecta o un disco duro está lleno.

Por el contrario, las **excepciones no controladas** (como `IllegalArgumentException`) se utilizan para **errores de lógica o violación de contratos**. Si un método recibe un parámetro inválido, suele ser un fallo del programador que llamó al método, no un problema del entorno. No se obliga a capturarlas porque la solución no es "gestionar el error", sino corregir el código fuente para que el error no vuelva a producirse.

En cuanto a otros lenguajes, **Java es prácticamente el único** lenguaje de uso masivo que implementa excepciones controladas. Lenguajes modernos como C#, Python, C++, Kotlin o Swift han decidido omitirlas.

En estos lenguajes, todas las excepciones se comportan como las **no controladas** de Java: el compilador nunca te obliga a poner un **try-catch**, dejando la responsabilidad de la robustez totalmente en manos del programador.

La opción **más habitual** en la industria es, por tanto, el modelo de **excepciones no controladas**. La experiencia en el desarrollo de software a gran escala demostró que obligar a capturar excepciones (el modelo de Java) a menudo lleva a los programadores a escribir bloques **catch** vacíos solo para silenciar al compilador, lo cual es más peligroso que no capturarlas. Por ello, la tendencia actual es confiar en excepciones voluntarias y una buena documentación.

16. ¿Tiene sentido lanzar excepciones dentro del **catch**? ¿Se puede relanzar la misma excepción capturada? ¿Cuándo tendría sentido hacer esto último? Pon ejemplos de ambos casos.

Lanzar una excepción dentro de un bloque **catch** tiene total sentido y es una práctica común en el diseño de software profesional. Se utiliza principalmente para realizar una **traducción de excepciones**. Esto ocurre cuando una función captura un error técnico de bajo nivel (como un **SQLException** de una base de datos) y lo transforma en una excepción que tenga más significado para la lógica del negocio (como un **UsuarioNoEncontradoException**), ocultando los detalles de implementación innecesarios.

Por otro lado, también es posible **relanzar la misma excepción** que se acaba de capturar. En este caso, el bloque **catch** actúa como un "punto de observación": el programa intercepta el error para realizar una acción secundaria (como registrar el fallo en un archivo de log o imprimir una traza), pero no intenta solucionar el problema. Al usar la palabra **throw** con el mismo objeto capturado, la excepción continúa su viaje original por la pila de llamadas.

Ejemplo 1: Lanzar una nueva excepción (Traducción)

En este caso, se captura un error genérico y se lanza uno específico. Es habitual pasar la excepción original como "causa" para no perder el rastro del error inicial.

```
try {  
    // Código que intenta leer un archivo de configuración  
    abrirFicheroConfig();  
} catch (FileNotFoundException e) {  
    // Traducimos un error técnico a uno de lógica de nuestra app  
    throw new ConfiguracionInvalidaException("Falta el archivo esencial de  
inicio", e);  
}
```

Ejemplo 2: Relanzar la misma excepción

Aquí el objetivo es solo dejar constancia del error antes de que el programa se detenga o lo gestione alguien más arriba.

```
try {
    double res = calc.dividir(a, b);
} catch (ArithmeticException e) {
    System.err.println("Log: Se intentó una división por cero en el módulo de facturación.");
    throw e; // Relanzamos la misma excepción para que el main decida qué hacer
}
```

17. ¿En qué consiste que una excepción sea la "**causa**" de otra excepción? Pon un ejemplo en Java, donde capturemos una excepción de bajo nivel y la encapsulemos en otra personalizada de alto nivel. Cuando una excepción sale por pantalla y tiene una causa, ¿se ve?

La **causa** de una excepción es un mecanismo que permite encadenar errores para no perder el rastro del fallo original cuando realizamos una traducción de excepciones. Consiste en pasar el objeto de la excepción de "bajo nivel" (la que ocurrió primero) como un argumento al constructor de una nueva excepción de "alto nivel". De este modo, la nueva excepción actúa como un envoltorio que mantiene la jerarquía de lo sucedido.

En términos de encapsulación, esto es fundamental: permite que las capas superiores de la aplicación manejen errores que entienden (como `ErrorAlProcesarPedido`), pero sin destruir la evidencia técnica (como un `NullPointerException` interno). Sin la "causa", al lanzar una nueva excepción estaríamos borrando la información de dónde empezó realmente el problema, lo que haría la depuración casi imposible en sistemas complejos.

```
// Definimos una excepción de alto nivel (negocio)
class ErrorPedidoException extends Exception {
    public ErrorPedidoException(String mensaje, Throwable causa) {
        super(mensaje, causa); // Pasamos la causa al constructor del padre
    }
}

public class Procesador {
    public void procesar() throws ErrorPedidoException {
        try {
            String dato = null;
            dato.length(); // Esto lanza un NullPointerException (bajo nivel)
        } catch (NullPointerException e) {
            // Capturamos el error técnico y lo envolvemos en uno de negocio
            throw new ErrorPedidoException("Fallo al procesar el pedido del cliente", e);
        }
    }
}
```

Cuando una excepción con causa sale por pantalla (por ejemplo, mediante `e.printStackTrace()`), **sí se ve claramente**. Java imprime primero la excepción principal y, justo debajo, añade una sección que comienza

con la frase "**Caused by:**" seguida del nombre, mensaje y traza de la excepción original.

Esto permite que un desarrollador vea el flujo completo: "Ocurrió un `ErrorPedidoException` causado por un `NullPointerException` en la línea X". Esta visualización es una de las herramientas más potentes de Java frente a C, ya que reconstruye la historia completa del fallo de forma automática, permitiendo identificar la raíz del problema en segundos.

¿Te gustaría que viéramos cómo personalizar aún más nuestra excepción para que guarde, por ejemplo, el ID del pedido que falló?